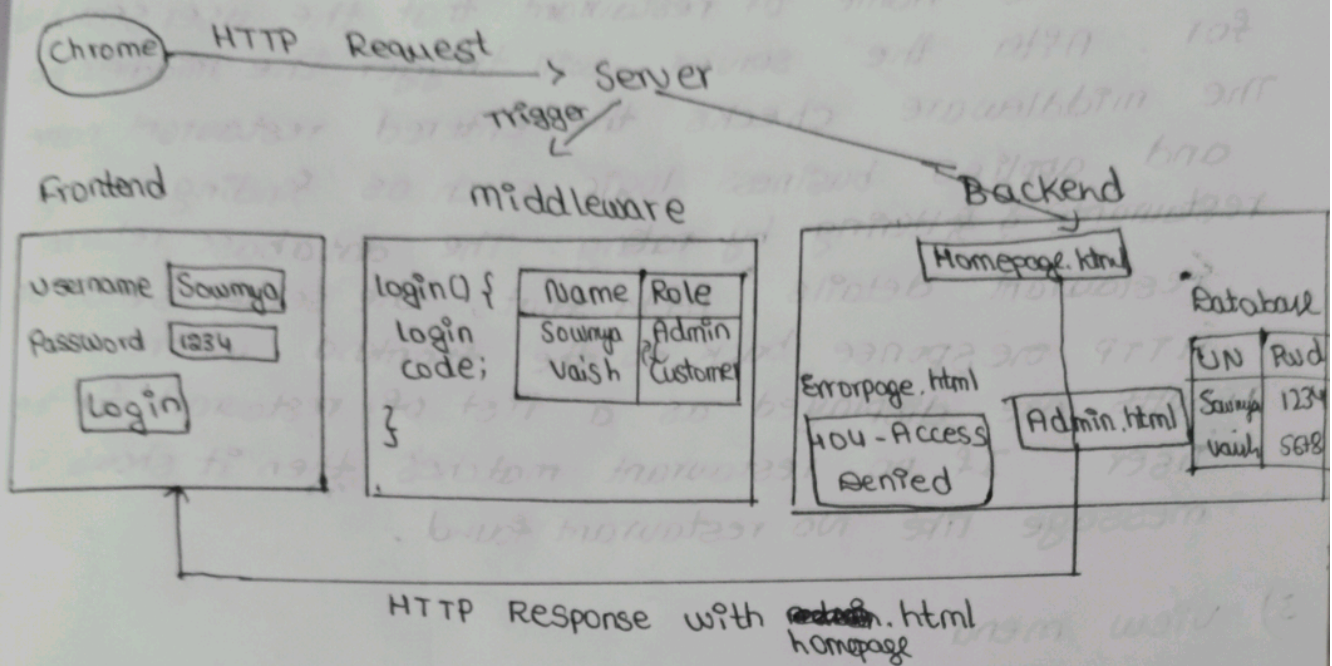


→ 3 tier architecture for Tomato application.

1) Login

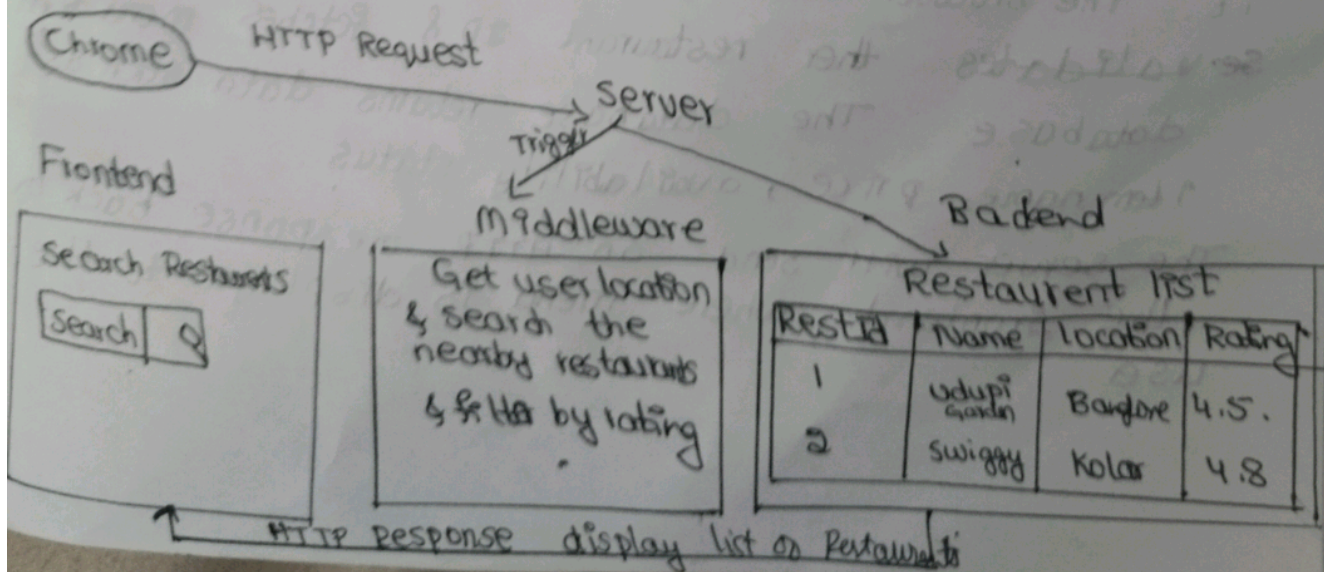


Frontend - user enter his username & password & then click the login button. Then the browser will send http request to server. Server triggers the middleware.

Middleware: It works as a business logic layer. It verifies username and password with data that is stored in backend.

Backend: If the details are correct the server will send HTTP Response with ~~Admin~~.html to the browser. If details are wrong the server will display 404 Access denied in the login page.

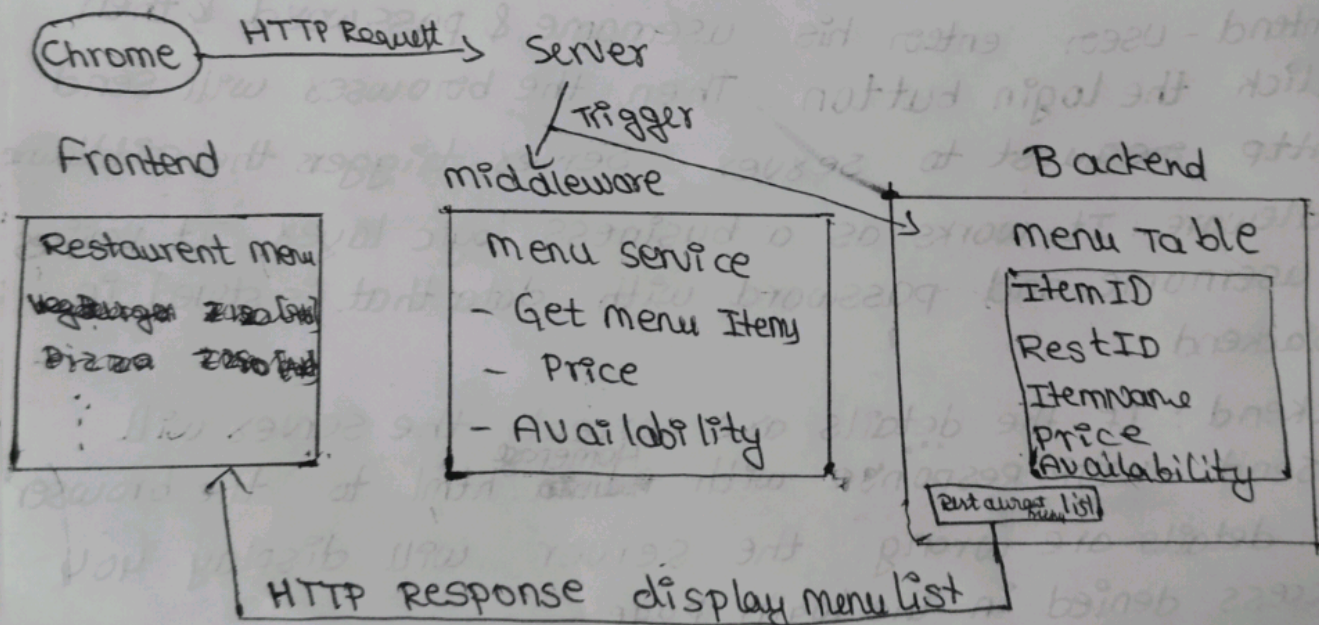
2) Restaurant Search



When the user logs in successfully & searches for a restaurant.

The browser sends an HTTP request to the server with the name of restaurant that the user searched for. After the server will trigger the middleware. The middleware checks the entered restaurant name and applies business logic such as finding nearby restaurants & filtering by rating. The database returns restaurant details. After that, the server sends an HTTP response back to the frontend where results are displayed as a list of restaurants for the user. If no restaurant matches then it shows a message like no restaurant found.

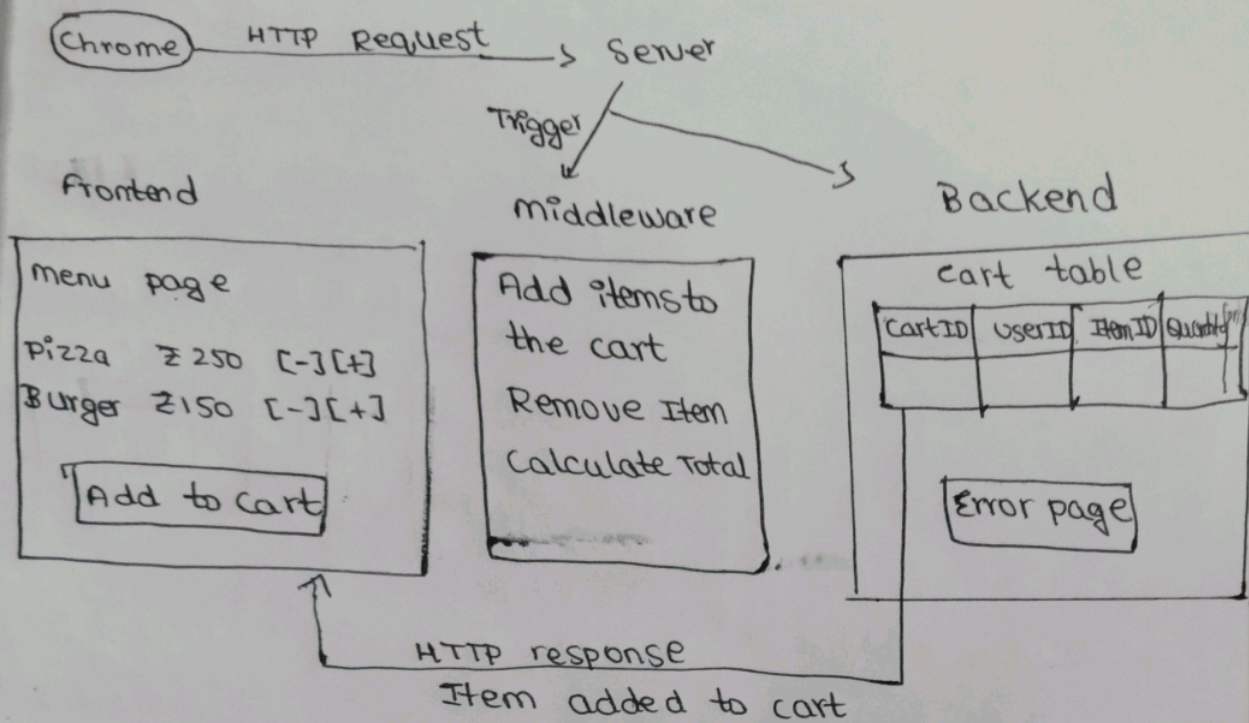
3) view menu



When the user selects a restaurant from search results & clicks on it. The browser sends HTTP request to the server. The server validates the restaurant ID & fetches menu from database. The database returns data such as item name, price, availability status.

The server will send an HTTP response back to the frontend where menu is displayed to the user.

4) Add to cart



When user selects item from menu & clicks Add to cart. The browser send HTTP request to the server with item details. The server triggers middleware, where cart & services handle tasks like adding, updating quantity, removing items if needed. The updated cart data is then stored in cart table in the database. Then server sends HTTP response back to the frontend, where the updated cart page is displayed to the user.

5) Payment

